

MODULE 21

Reinforcement Learning

AI with Data Science - Professional Learning Series

ARSHITH LMS | Professional Edition

Reinforcement Learning

What is Reinforcement Learning?

RL: agent learns by interacting with environment.

Key components:

Agent: the learner/decision maker

Environment: what the agent interacts with

State: current situation

Action: what the agent does

Reward: feedback signal

Policy: strategy mapping states to actions

Applications: games, robotics, trading, self-driving

Key RL Concepts

Exploration vs Exploitation:

Explore: try new actions to learn

Exploit: use best known action for reward

Epsilon-Greedy strategy:

With prob epsilon: explore (random action)

With prob 1-epsilon: exploit (best action)

Return (G): sum of discounted future rewards

$G = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots$

gamma (discount factor): 0=short-sighted, 1=far-sighted

Q-Learning

```
import numpy as np

# Q-table: state x action values
Q = np.zeros((state_size, action_size))

for episode in range(1000):
    state = env.reset()
    done = False
    while not done:
        # Epsilon-greedy action selection
        if np.random.random() < epsilon:
            action = env.action_space.sample() # explore
        else:
```

```

action = np.argmax(Q[state]) # exploit
next_state, reward, done, _ = env.step(action)
# Q-learning update
Q[state, action] += alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state, action]
)
state = next_state

```

Deep Q-Network DQN

```

import tensorflow as tf
import numpy as np
from collections import deque

class DQNAgent:
    def __init__(self, state_size, action_size):
        self.memory = deque(maxlen=2000)
        self.gamma = 0.95
        self.epsilon = 1.0
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995
        self.model = self._build_model(state_size, action_size)

    def _build_model(self, state_size, action_size):
        model = tf.keras.Sequential([
            tf.keras.layers.Dense(64, activation='relu', input_shape=(state_size,)),
            tf.keras.layers.Dense(64, activation='relu'),
            tf.keras.layers.Dense(action_size, activation='linear')
        ])
        model.compile(optimizer='adam', loss='mse')
        return model

```

OpenAI Gymnasium

```

import gymnasium as gym
import numpy as np

# Create environment
env = gym.make('CartPole-v1', render_mode='human')

for episode in range(10):
    obs, info = env.reset()
    total_reward = 0
    done = False
    while not done:
        # Random policy
        action = env.action_space.sample()
        obs, reward, terminated, truncated, info = env.step(action)
        done = terminated or truncated
        total_reward += reward
    print(f'Episode {episode}: reward={total_reward}')

env.close()

```

